

Kafka with spring boot

Introduction to Apache Kafka

When we create any small application:

- we deal with small amount of data
- few users use the application
- one application talks to few components
- everything looks simple

For example:

- one Spring Boot application
- one database
- few API calls

At this stage, communication is easy and manageable.

In the real world, applications do not work with small data. There are many applications that work with **millions of users every day**.

Examples:

- IRCTC
- Amazon
- Flipkart
- Swiggy
- Paytm

These applications receive **millions of requests daily** Because of this:

- millions of records are generated
- millions of payments happen
- millions of messages are sent

Let's Take One Train Ticket Example

Every day:

- millions of users book train tickets
- millions of payments are processed
- millions of SMS are sent
- millions of emails are sent

Now think carefully.

- Booking is one thing
- Payment is another thing
- Notification is another thing

All these are **different components**, but they must **talk to each other**.

What Actually Happens Behind the Scene When a ticket is booked:

1. booking component creates ticket
2. payment component processes payment
3. notification component sends SMS and email
4. logging system stores logs

So one action creates **multiple actions**. This happens **millions of times every day**

The challenge is not booking a ticket. The real challenge is **How do all these components communicate with each other without slowing the system?**

If all components call each other directly:

- system becomes tightly connected
- failure in one component affects others
- system becomes slow
- scaling becomes very difficult

This approach works only for small systems

In large systems:

- components should work independently
- main flow should be fast
- extra work should happen separately

So instead of calling components directly, we **send information** about what happened **This Information Is Called a Message**

A message means: **Something happened in the system.**

Examples:

- "Ticket booked"
- "Payment successful"
- "Send notification"

Once the message is sent:

- other components can react to it
- main flow does not wait

What Is a Message Broker?

A message broker is a middle system that transfers messages from one component to many other components.

system that can:

- receive messages
- store messages
- deliver messages

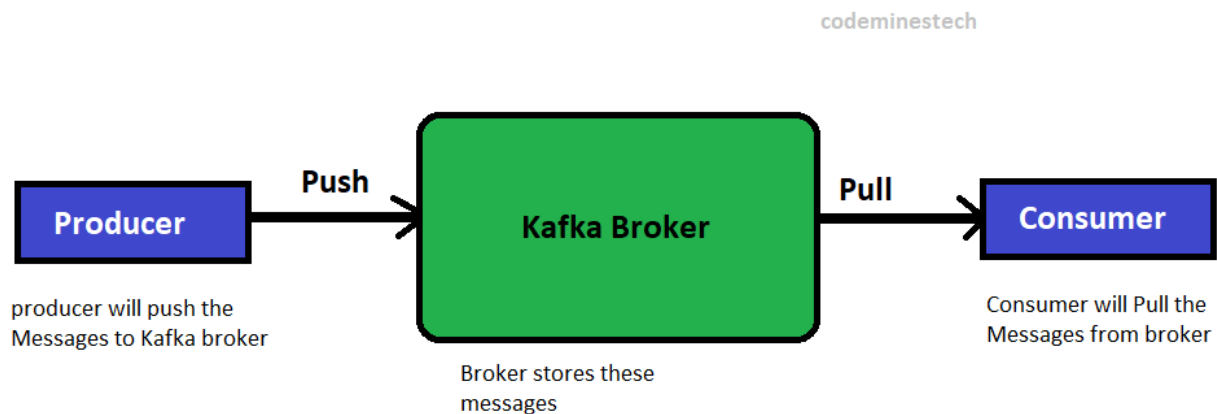
This system is called a **message broker**.

Where Apache Kafka Comes In

Apache Kafka is a distributed event streaming platform used to handle large volumes of messages in real time.

Apache Kafka is a **powerful message broker** designed for:

- very large data
- very high traffic
- millions of messages per day



Kafka is needed because:

- real-world systems generate huge data
- many components need same information
- systems must not depend on each other
- data should not be lost

Kafka solves all these problems.

Apache Kafka is a software system that is used to manage communication between different parts of an application or between different applications. It works as a central platform where data is sent, stored, and shared. Kafka does not work on a single machine only. It is designed to run on multiple machines at the same time. This distributed nature helps Kafka handle very large amounts of data and also keeps the system running even if one machine fails.

Kafka works with events. An event represents something that has happened in the system, such as a ticket being booked, a payment being completed, or a notification being sent. Every time such an action happens, information related to that action is created. Kafka receives this information in the form of messages and manages it efficiently.

Kafka is called an event streaming platform because it handles events as a continuous flow of data. In real-world applications, events are generated all the time by users and systems. Kafka processes this data continuously instead of handling it one message at a time. This makes Kafka suitable for applications where data is generated continuously and needs to be processed without delay.

Kafka is also called a platform because it provides more than just message transfer. It stores messages safely, allows multiple consumers to read the same data, and ensures that messages are delivered reliably. Kafka is specially designed to handle very large volumes of messages. It can manage thousands or millions of messages without becoming slow.

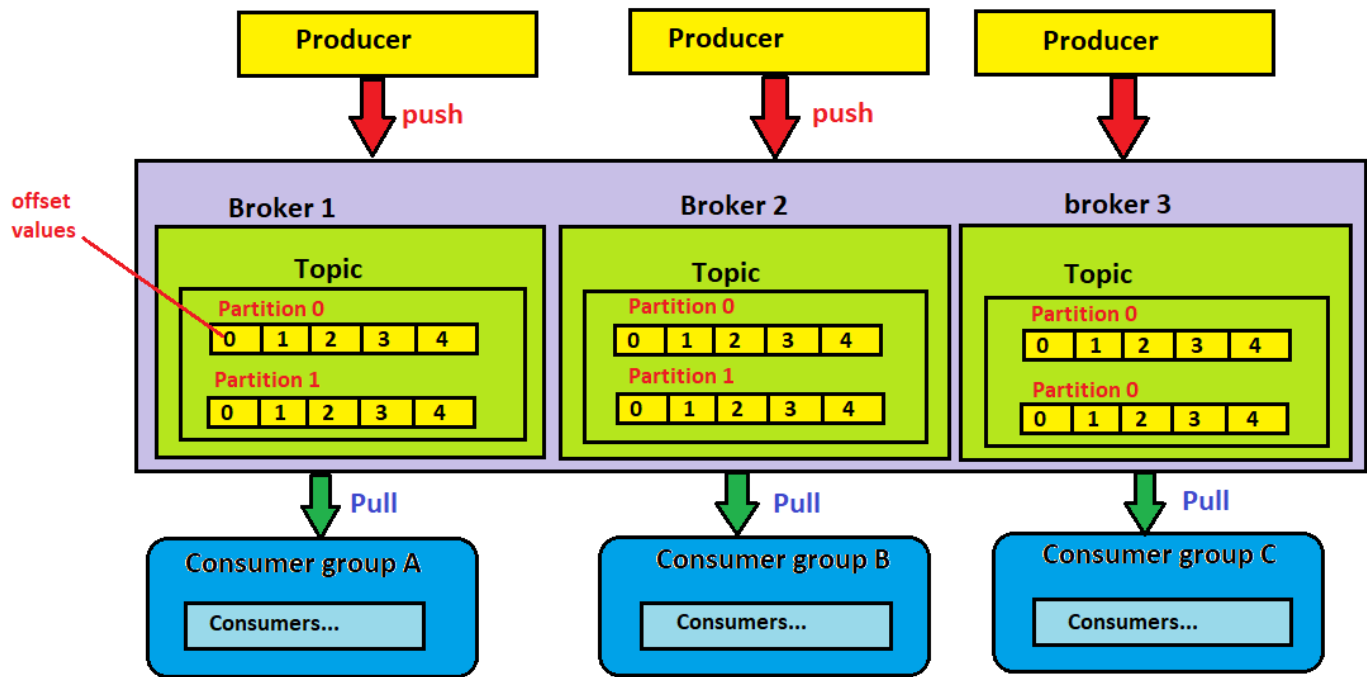
Kafka works in real time, which means that as soon as an event occurs, the data related to that event is immediately available to the systems that need it. This real-time processing is very important for applications like online booking systems, payment systems, notifications, and analytics, where quick response is required.

In simple words, Apache Kafka is a distributed system that continuously processes large amounts of event data and makes it available in real time to different parts of an application or to different applications.

Apache Kafka – Architecture and Core Concepts

Apache Kafka is designed as a distributed system to handle very large amounts of data reliably and efficiently. To understand Kafka properly, it is important to understand how its architecture is structured and how different components work together. Kafka is not a single program or a single server. It is a complete system made of multiple parts, where each part has a clear responsibility.

At a high level, Kafka works on the idea that data is produced by some systems, stored safely in Kafka, and then consumed by other systems. This data flow is continuous and happens at a very large scale.



Kafka Cluster and Brokers

A Kafka system is called a **Kafka cluster**. A cluster means a group of machines working together. In Kafka, each machine in the cluster is called a **broker**. A broker is simply a Kafka server that runs Kafka software and stores data.

Kafka is designed to run on multiple brokers instead of a single server because real-world systems generate huge amounts of data. If Kafka worked on only one server, that server could become slow or fail. By running Kafka on multiple brokers, the load is divided and the system becomes more reliable.

Each broker has its own storage and memory. Brokers communicate with each other and together they form the Kafka cluster. From the application point of view, producers and consumers do not talk to a specific broker manually. They connect to the Kafka cluster, and Kafka internally decides which broker should handle the request.

Topics in Kafka

In Kafka, data is organized using **topics**. A topic is a logical name under which messages are stored. You can think of a topic as a category or a stream of related data.

For example, in an Train booking like system:

- one topic can be used for booking events

- another topic for payment events
- another topic for notification events

Topics help Kafka organize data so that producers know where to send messages and consumers know from where to read messages. A topic does not store data by itself. Internally, a topic is divided into smaller units called partitions.

Partitions and Their Purpose

Each topic in Kafka is divided into one or more **partitions**. Partitions are the core unit of data storage and parallelism in Kafka. When data is sent to a topic, Kafka actually writes that data into one of the topic's partitions.

Partitions exist for two main reasons. First, they allow Kafka to handle large amounts of data by splitting the data across multiple brokers. Second, they allow multiple consumers to read data in parallel, which improves performance.

Offset and Message Ordering

Inside each partition, messages are stored in a strict order. Kafka assigns a unique number to each message in a partition. This number is called an **offset**. The offset represents the position of a message inside the partition.

Offsets are important because Kafka does not track messages by deleting them after consumption. Instead, consumers keep track of which offset they have already read. This design makes Kafka very efficient and allows consumers to re-read messages if needed.

Producers in Kafka

A **producer** is any application that sends data to Kafka. Producers generate events or messages and publish them to Kafka topics. Producers do not send data directly to consumers. They only send data to Kafka.

When a producer sends a message, Kafka decides which partition the message should go to. This decision can be based on a key, or Kafka can choose a partition automatically. Producers push data into Kafka, and once the data is written successfully, Kafka takes responsibility for storing it safely.

Consumers in Kafka

A **consumer** is an application that reads data from Kafka. Consumers do not receive data automatically. Instead, consumers **pull** data from Kafka. This means a consumer asks Kafka for new messages when it is ready to process them.

Consumers read data from topics and partitions. Each consumer keeps track of the offsets it has already processed. This allows consumers to stop, restart, or fail without losing data.

Kafka consumers are usually grouped together into **consumer groups**. A consumer group is a logical group of consumers that work together to process data from a topic. Consumer groups are the main reason Kafka scales so well. If the data load increases, more consumers can be added to the group. Kafka will automatically rebalance partitions among consumers.

How Everything Works Together

In Kafka architecture, producers send messages to topics. Topics store messages in partitions across multiple brokers. Consumers pull messages from these partitions and process them. Offsets help consumers track progress, and consumer groups help Kafka scale message consumption.

This design allows Kafka to handle huge volumes of data, support multiple consumers, and remain reliable even if some brokers fail.

Replication, Leader–Follower and Fault Tolerance in Apache Kafka

Apache Kafka is designed to work in real-world systems where failures are normal. Servers can crash, disks can fail, and networks can go down. Kafka assumes that failures will happen, so it is built in a way that data is not lost even when something goes wrong. This reliability comes mainly from replication and the leader–follower mechanism.

In Kafka, data is stored in partitions, and each partition can have multiple copies. These copies are called **replicas**. Replication means that the same data is stored on more than one broker. Kafka does not keep all data on a single server. Instead, it spreads data across multiple brokers so that if one broker fails, another broker still has the data.

For every partition, Kafka chooses one broker as the leader. The leader is responsible for handling all read and write operations for that partition. When a producer sends a message,

it always writes the message to the leader partition. Consumers also read data only from the leader. This design keeps the system simple and consistent because there is only one source of truth for reads and writes.

The other copies of the partition are called followers. Followers do not handle client requests directly. Their job is to continuously copy data from the leader. This copying process is called replication. As new messages are written to the leader, the followers replicate the same messages in the same order. In this way, followers always try to stay in sync with the leader.

Replication is important because it provides fault tolerance. If the broker that hosts the leader partition crashes or becomes unavailable, Kafka automatically selects one of the in-sync followers as the new leader. This process is handled internally by Kafka and does not require manual intervention. Once a new leader is elected, producers and consumers continue working with minimal interruption.

Kafka keeps track of which followers are fully up to date with the leader. These followers are called in-sync replicas. Only in-sync replicas are eligible to become leaders. This ensures that when a leader fails, the new leader always has the latest data. Because of this design, Kafka can guarantee that committed data is not lost even during failures.

From the user's point of view, this entire process is mostly invisible. Producers continue to send data, and consumers continue to read data without needing to know which broker is currently acting as the leader. Kafka handles leader changes and replication internally, which makes it suitable for large-scale, always-on systems.

Replication also helps Kafka scale. Since data is distributed across multiple brokers, disk usage and network load are shared. This allows Kafka to handle very large volumes of data without overloading a single machine. At the same time, replication ensures that the system remains reliable and available.

In simple terms, replication and the leader–follower model are the backbone of Kafka's reliability. They ensure that data is always available, failures are handled automatically, and large systems can continue working without data loss. This is one of the main reasons Kafka is trusted in production systems where data is critical.

ZooKeeper in Apache Kafka

In Apache Kafka 3.x and earlier versions, ZooKeeper is an important component that is used to manage and coordinate the Kafka cluster. Kafka itself is a distributed system with multiple brokers working together. When many servers work together, they need a central system to manage metadata and coordination. ZooKeeper provides this coordination service for Kafka.

ZooKeeper is a separate system that runs independently from Kafka brokers. Kafka brokers connect to ZooKeeper to share and receive information about the cluster. ZooKeeper does not store actual Kafka messages or data. Instead, it stores metadata, which means information about the Kafka system itself.

One of the main responsibilities of ZooKeeper is cluster management. ZooKeeper keeps track of all Kafka brokers that are currently running. When a broker starts, it registers itself with ZooKeeper. When a broker stops or crashes, ZooKeeper detects this change and updates the cluster information. This allows Kafka to always know which brokers are alive and which are not.

ZooKeeper also plays a key role in leader election. As discussed earlier, each Kafka partition has a leader and one or more followers. ZooKeeper helps Kafka decide which broker should act as the leader for each partition. When a leader broker fails, ZooKeeper helps Kafka select a new leader from the available in-sync replicas. This process ensures that the system continues to work without data loss.

Another important responsibility of ZooKeeper is managing configuration and metadata. ZooKeeper stores information such as topics, partitions, replication details, and access control information. Kafka brokers read this information from ZooKeeper to understand how the cluster is configured. This allows all brokers in the cluster to have a consistent view of the system.

In Kafka 3.x, ZooKeeper is required because Kafka relies on it for coordination, metadata storage, and leader election. However, managing ZooKeeper adds operational complexity because ZooKeeper itself is a distributed system that must be installed, configured, and monitored separately.

In Kafka 4.x and later, Kafka manages cluster metadata internally without ZooKeeper. This makes Kafka easier to operate while keeping the same core concepts.

Working with Apache Kafka 3.x

In Apache Kafka 3.x, Kafka uses **ZooKeeper** for cluster coordination. So before working with Kafka brokers, **ZooKeeper must be started first**. Kafka brokers depend on ZooKeeper to manage metadata, broker information, and leader election.

Below is some step to work with Kafka 3.x on a local machine (Windows).

Download and Extract Kafka

First, download Apache Kafka **3.x version** from the official Apache Kafka website.

After downloading:

- extract the Kafka zip file
- open the extracted folder
- all commands should be run from this Kafka root directory

Kafka provides ready-to-use scripts inside the **bin/windows** folder.

Start ZooKeeper :- use command

`.\bin\windows\zookeeper-server-start.bat .\config\zookeeper.properties`

ZooKeeper works as the **cluster manager** in Kafka 3.x.

It keeps track of:

- brokers
- topics
- partitions
- leaders

Keep this command window open. Do not close it while Kafka is running.

Start Kafka Broker (Single Broker)

`.\bin\windows\kafka-server-start.bat .\config\server.properties`

By default:

- broker runs on port **9092**
- broker registers itself with ZooKeeper

Start Multiple Kafka Brokers (Kafka Cluster) if needed

```
.\\bin\\windows\\kafka-server-start.bat .\\config\\server.properties
```

```
.\\bin\\windows\\kafka-server-start.bat .\\config\\server-1.properties
```

```
.\\bin\\windows\\kafka-server-start.bat .\\config\\server-2.properties
```

Each command starts a separate broker. Together, these brokers form a **Kafka cluster**. ZooKeeper coordinates all these brokers.

List All Topics

```
.\\bin\\windows\\kafka-topics.bat --list --bootstrap-server localhost:9092
```

This command shows:

- all existing topics
- helps verify cluster connectivity

Create a Topic (Simple)

```
.\\bin\\windows\\kafka-topics.bat --create --topic topicName --bootstrap-server localhost:9092
```

This creates:

- 1 partition

Create a Topic with Partitions and Replication

```
.\\bin\\windows\\kafka-topics.bat --create --topic topicName --partitions 3 --replication-factor 3 --bootstrap-server localhost:9092
```

This means:

- topic has 3 partitions
- each partition has 3 replicas
- data is distributed across brokers

Describe a Topic

```
.\bin\windows\kafka-topics.bat --describe --topic topicName --bootstrap-server localhost:9092
```

This command shows:

- partition count
- leader broker
- replicas
- in-sync replicas (ISR)

Very useful for understanding Kafka internals.

Start Producer (Send Messages)

```
.\bin\windows\kafka-console-producer.bat --topic topicName --bootstrap-server localhost:9092
```

After running this command:

- type messages
- press Enter to send
- each line becomes a Kafka message

Producer **pushes data** into Kafka.

Start Consumer (Read All Messages)

```
.\bin\windows\kafka-console-consumer.bat --topic topicName --bootstrap-server localhost:9092 --from-beginning
```

This command:

- reads messages from all partitions
- starts reading from the beginning

Useful for verifying produced messages.

Read from a Specific Partition

```
.\.bin\windows\kafka-console-consumer.bat --topic topicName --partition 0 --from-beginning --bootstrap-server localhost:9092
```

This command:

- reads only partition 0
- helps understand partition-wise data